From CLAUDE.md to 496 PRs: Progressively Mastering Every Claude Code Mechanism

One developer. Two production applications. **1,180 commits** in under a month — not by coding faster, but by stacking every Claude Code mechanism into an autonomous development machine.

AI-DRIVEN DEVELOPMENT

DEVELOPER ARCHITECTURE

REAL PRODUCTION DATA

One Developer, 496 PRs, 1,180 Commits

The Scale

1,180 commits in under a month

496 merged PRs across two apps

160 issues resolved autonomously

94% autonomous story completion

The Method

Not by coding faster — by progressively stacking every Claude Code mechanism into an autonomous development machine.

CLAUDE.md → Agents → Skills → Commands → Hooks → Worktrees → cmux → GitHub at every step.

This deck shows how each mechanism was adopted, why it matters, and the real numbers it produced.

1,180

Total Commits

Across both projects

496

Merged PRs

Every one reviewed

160

Issues Resolved

Autonomously fixed

55

Epics Delivered

End-to-end

The Journey: From Single-File Config to Full Orchestration

The system wasn't built in a day. Each Claude Code mechanism solved a specific pain point — and they compound on each other. Remove any layer and the system degrades.

1

Layer 1

CLAUDE.md — Consistent standards across sessions

2

Layer 2

Agents — Specialized expertise per domain

3

Layer 3

Skills — Repeatable multi-step workflows

4

Layer 4

Commands — Quick-access operations

5

Layer 5

Hooks — Automated lifecycle events

6

Layer 6

Worktrees — Parallel agent isolation

7

Layer 7

cmux bridge — Real-time observability

8

Layer 8

GitHub (gh CLI) — Version control at every step

Layer 1: CLAUDE.md — The Constitution Every Agent Reads

CLAUDE.md is the foundation. It's the first file every Claude Code session loads — the global instructions that shape all behavior.

What Ours Enforces

- Address the developer as "FX" — sharp, no-nonsense tone
- TDD always (skip requires explicit authorization string)
- Never use `--no-verify` on commits
- Python uses `uv` + FastAPI; TypeScript uses Bun runtime
- Use `fd` over `find`, `bat` over `cat`, `scc` for LOC, `typst` for PDF
- All GitHub operations via `gh` CLI (no MCP)

Why It Matters

Without CLAUDE.md, every session starts from zero. With it, every agent inherits the same standards, tooling preferences, and quality bar — **automatically**.

> CLAUDE.md is not optional. It's the DNA of the entire system. Every agent, every skill, every command inherits from it.

Layer 2: Agents — 12 Specialized Experts, Not Generic Assistants

Instead of one generalist, we defined **12 specialized agents** — each with a system prompt, tool restrictions, and domain expertise. The orchestrator reads each story's tags and tech stack, then dispatches the right agent automatically.

Agent	Domain	Key Expertise
backend-typescript-architect	Bun, APIs, microservices	Database optimization, auth, N+1 prevention
python-backend-engineer	FastAPI, SQLAlchemy, async	Pydantic models, uv dependency management
ui-engineer	React, components, accessibility	CSS-in-JS, state management, responsive design
bash-zsh-macos-engineer	Shell, CI/CD, DevOps	macOS automation, launchd, Homebrew
podman-container-architect	OCI, multi-stage builds	Rootless Podman, Python+uv container patterns
qa-engineer	Test strategy, coverage	Defect management, automation >70% target
senior-code-reviewer	Architecture, security	15+ years experience, OWASP, mandatory gate

Auto-selection: FastAPI → python-backend-engineer. React → ui-engineer. Code review → senior-code-reviewer, always.

Layer 3: Skills — 16 Repeatable Workflows That Orchestrate Agents

Skills are multi-step workflows that chain agents, tools, and quality gates into reproducible pipelines. Each skill declares `allowed-tools`, `argument-hint`, and supporting files.

01

`/project-init`

Bootstrap repo with CLAUDE.md, PROJECT-SEED.md, CI/CD config

02

`/brainstorm`

PM-style 8-question interview → REQUIREMENTS.md

03

`/generate-epics`

Requirements → AGILE epic files with INVEST-compliant stories

04

`/build-stories`

Multi-story autonomous build with TDD, coverage, review, merge

05

`/create-issue`

Structured GitHub issue creation with severity and labels

06

`/fix-issue`

11-phase autonomous pipeline from investigation to merge

Layer 4: Commands — 17 Slash Commands for Quick Operations

Commands are lightweight, single-purpose actions organized into **6 categories**. Skills orchestrate multi-step workflows with agents and quality gates. Commands execute a focused single action. Both invoked with `/`, but they serve different scales of work.

Quality (3)

- `/quality/roast` — brutal code review
- `/quality/coverage`
- `/quality/project-review`

Project (5)

- `/project/update-progress`
- `/project/create-project-summary-stats`
- `/project/create-user-documentation`

Issues (1)

- `/issues/create-issue` — comprehensive defect analysis

DevOps (2)

- `/devops/check-releases`
- `/devops/plan-release-update`

Research (3)

- `/research/crypto-analysis`
- `/research/profile-analysis`
- `/research/client-analysis`

Generators (1)

- `/backfill-docs` — documentation for existing code

Layer 5: Hooks — 7 Automated Lifecycle Events

Hooks fire automatically on Claude Code lifecycle events — no manual trigger needed. The PR merge hook is especially powerful: every time a PR merges, it injects context that prompts Claude to update README, story progress, and project docs.

Event	Hook	What It Does
SessionStart	`cmux-session-start.sh`	Sets blue "Ready" status pill
SubagentStart	`cmux-agent-start.sh`	Shows "Running: {agent_type}" pill
SubagentStop	`cmux-agent-stop.sh`	Clears pill, logs completion, desktop notification
Stop	`cmux-stop.sh`	Clears all progress, "Claude Done" notification
Notification	`cmux-permission.sh`	Red "Permission Needed" pill + urgent alert
PostToolUse	`on-pr-merge-docs.sh`	After → triggers doc updates

Sentinel pattern: `/build-stories` writes `/tmp/.claude-skill-active` so agent-level hooks stay quiet during batch runs — only the orchestrator reports.

Layer 6: Git Worktrees — Parallel Agents Without File Conflicts

The breakthrough for parallel execution. **Git worktrees** give each agent an isolated directory tree while sharing the same `.git` directory.

Without Worktrees

Agents overwrite each other's files. One at a time only. Sequential bottleneck kills throughput.

With Worktrees

Up to **5 agents build concurrently**. Branches are visible across worktrees because they share `.git`.

Concurrency Model

Stage	Agent	Isolation	Concurrency
Build (TDD)	Specialized	Worktree	Up to 5
Coverage gate	qa-engineer	Worktree	Up to 5
Code review	senior-reviewer	Read-only	Up to 5
Merge	Orchestrator	Sequential	1

Critical rule: Build agents always `git push` before returning — the coverage agent in another worktree needs to `git fetch` the branch.

Layer 7: cmux Bridge — Real-Time Observability

cmux is a native macOS terminal built for multi-agent AI development. The bridge script (105 lines) provides **8 subcommands** for complete visibility into what every agent is doing.

Status Pills

Color-coded phase indicators: blue=running, green=done, red=failed

Progress Bars

Fraction complete 0.0 → 1.0, updated per phase or per story

Sidebar Logs

Permanent event ledger: agent started, coverage %, build completion

Desktop Notifications

Milestones only: preflight failure, E2E gate results, build completion

Telegram Fallback

Envelope alerts via Bot API (start, first failure, abort, finish). Rate-limited.

Pane Management

Parallel mode auto-creates split panes per cohort, labeled with story IDs

Graceful degradation: If cmux isn't running, every hook silently no-ops. The workflow is identical — you just don't see the sidebar.

Layer 8: GitHub at Every Single Step

GitHub isn't an afterthought — it's the backbone. The `gh` CLI is used at **every step** of the workflow. Full traceability: every PR links to a story ID, every issue links to a fix branch, every merge triggers documentation updates.

Step	GitHub Operation	Tool
Pre-flight	Verify auth + clean state	`gh auth status`
Build	Create feature branch	`git checkout -b feature/{ID}`
Coverage	Push + create PR	`git push -u` → `gh pr create`
Review	Comment on PR	`gh pr review`
Merge	Merge + trigger hook	`gh pr merge` → `on-pr-merge-docs.sh`
Issue create	Structured issue	`gh issue create --label bug,high`
Issue fix	Fetch + fix + close	`gh issue view` → fix → `gh issue close`
E2E gate	Check PR status	`gh pr checks`
Progress	Link PRs to stories	PR numbers tracked in `.build-progress.md`

The Complete Pipeline: From Blank Repo to Production

Every phase produces GitHub artifacts — commits, branches, PRs, issues, reviews. Complete audit trail from idea to deployment.

1

Phase 1: Discovery

`/project-init` → `/brainstorm` → REQUIREMENTS.md committed to GitHub

2

Phase 2: Planning

`/generate-epics` → Epic files + STORIES.md committed

3

Phase 3: Build

`/build-stories all --parallel` → TDD → Coverage (90%+) → Review → PR → Merge

4

Phase 4: Fix

`/fix-issue all` → Investigate → TDD fix → Coverage → Review → Merge → Close

5

Phase 5: Quality

`/design-e2e` → `/execute-e2e-tests` → `/coverage` → `/project-review`

Deep Dive: /build-stories — The 7-Phase Orchestrator

The most complex skill. A **thin dispatcher** that never reads source code — only coordinates agents. Supports `--parallel`, `--sequential`, `--dry-run`, `--skip-coverage`, `--auto`, `--e2e-gate=block|warn|off`, `--limit=N`.

1. **Parse & Validate**
   Parse arguments, validate environment (clean git, `gh auth`, main branch)

2. **Discovery & Sort**
   Discovery Agent → topological sort → build queue (JSON)

3. **Queue Parsing**
   Queue parsing with `--limit=N` and dependency integrity checks

4. **Cohort Organization**
   Organize into dependency cohorts (no unmet deps = first cohort)

5. **Per-Story Loop**
   Build → Coverage → Review → Merge → Bugfix if needed

6. **Summary & Docs**
   Summary agent + documentation update agent

7. **Report & Notify**
   Report, notification, Telegram alert, cleanup sentinel

Deep Dive: /fix-issue — 11-Phase Autonomous Bug Resolution

Batch mode: `--limit=5` runs 5 issues in parallel with **file-overlap guard** — issues touching the same files are auto-serialized.

Phase	Progress	Action
1	0.09	Validate issue, check git, pre-flight tests
2	0.18	Fetch from GitHub via `gh issue view`
3	0.27	Investigation agent: root cause analysis
4	0.36	Build agent: TDD fix on `fix/issue-{N}` branch
5	0.45	Coverage gate: 90%+ or bugfix loop
6	0.64	Code review by senior-code-reviewer
7	0.73	E2E tests (if enabled)
8	—	Bugfix loop: CODE_BUG / TEST_BUG / ENV_ISSUE, max 2 retries
9	0.82	Merge PR + close issue via `gh`
10	0.91	Generate fix report
11	1.0	Desktop + Telegram notification

The Quality Gates: 4 Mandatory Checks, No Exceptions

1

TDD Cycle

Tests written first. Code implemented to pass. Then refactored. **Red → Green → Refactor** on every story.

2

Coverage (90%+)

qa-engineer verifies branch coverage. Below threshold triggers bugfix loop to add missing tests.

3

Code Review

senior-code-reviewer checks architecture, security (OWASP), performance, naming, test quality. Approves or requests changes.

4

E2E Tests

After all stories in an epic merge, Playwright end-to-end tests validate user workflows. Modes: `block`, `warn`, `off`.

Bugfix loop (max 2 retries): Classifies as CODE_BUG, TEST_BUG, or ENV_ISSUE. Creates GitHub issue for CODE_BUG. Auto-fixes and retests. If unfixed: mark FAILED, alert, continue.

MCP Servers & CLI Tools: The Supporting Infrastructure

MCP Servers (2)

Playwright — Browser automation for E2E testing (`/design-e2e`, `/execute-e2e-tests`)

Context7 — Documentation retrieval and context management

Templates Directory

Scaffolding for new agents, skills, and commands — the system extends itself.

9 CLI Tools Mandated in CLAUDE.md

Tool	Replaces	Benefit
`fd`	`find`	10x faster, respects .gitignore
`rg`	`grep`	Best-in-class search
`bat`	`cat`	Syntax highlighting + git integration
`zoxide`	`cd`	Frecent directory jumping
`yazi`	`ls`	Full terminal file manager
`jq`	—	JSON processing
`scc`	`wc -l`	Code counter with complexity
`typst`	reportlab	Modern typesetting → PDF
`pdftotext`	—	PDF text extraction

The Numbers: Two Projects, Real Production Code

Both are **full-stack applications**: FastAPI backends, React frontends, PostgreSQL, Docker, Alembic migrations, Playwright E2E tests. Built almost entirely by AI agents.

Metric	infobasic-bench	forge	Combined
Commits	508	672	**1,180**
Merged PRs	216	280	**496**
Issues resolved	93	67	**160**
Epics	30	25	**55**
Stories	—	241 (227 done)	**94% autonomous**
Languages	Python 64% / TS 31%	Python 65% / TS 35%	Full-stack
Timeline	~4 weeks	18 days	—
Agents defined	12	12	**12**
Skills	16	16	**16**
Commands	17	17	**17**
Hooks	7	7	**7**

What We Built: infobasic-bench & forge

INFOBASIC-BENCH

Core Banking Code Analysis Platform

Evaluates InfoBasic/jBC code quality across LLM models with scoring dimensions, cost tracking, and benchmark reports.

Used to assess **Temenos T24 modernisation** approaches. 508 commits, 216 PRs, 93 issues resolved.

- Multi-model LLM benchmarking
- Cost tracking per analysis run
- Automated benchmark report generation

FORGE

Full Temenos T24 Assessment Platform

Ingests client code extracts, maps dependencies, runs LLM analysis pipelines, generates migration specs and deliverables.

Multi-tenant with client/programme/instance hierarchy, RBAC, and a **glassmorphism design system**. 672 commits, 280 PRs.

- TAFJ reports, test cases, assessment reports
- Full RBAC + multi-tenant architecture
- Playwright E2E test suite

Both built almost entirely by AI agents — from Docker Compose to the last Playwright E2E test.